

UNIVERSITY OF SRI JAYEWARDENEPURA
FACULTY OF TECHNOLOGY
BACHELOR OF INFORMATION AND COMMUNICATION TECHNOLOGY

IIC 2223: Web Application Development
Laboratory Exercise 02: AI Web Application Challenge

Name: L. A. G. Nipulma
Index Number: ICT/24/896

Task 1: Investigate Browser-Server Communication

1.1 Network Observation Table

Target Website: <https://www.sjp.ac.lk>

Table:  WAP-Lab2-Tables (Network Observation Table)

1.2 Detailed Request Analysis

Selected Resource:

<https://www.sjp.ac.lk/wp-content/plugins/contact-form-7/includes/css/styles.css?ver=6.1.4>

1. URL Breakdown

- **Scheme:** https
- **Domain:** www.sjp.ac.lk
- **Port:** 443 (default port for HTTPS)
- **Path:** wp-content/plugins/contact-form-7/includes/css/styles.css
- **Query String:** ver=6.1.4

2. Request vs Response

- **Request Method:** GET
- **Request URL:**
[https://www.sjp.ac.lk/wp-content/plugins/contact-form-7/includes/css/styles.css?ver=6.1.4]
- **Request Headers:** HTTP/1.1
Host: www.sjp.ac.lk
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:156.0) Gecko/20100101 Firefox/156.0
Accept: text/css;q=0.1
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br, zstd
Referer: <https://www.sjp.ac.lk/>
Sec-GPC: 1
Connection: keep-alive
Sec-Fetch-Dest: style
Sec-Fetch-Mode: no-cors
Sec-Fetch-Site: same-origin
- **Request Body:** None
- **Response Status:** 200
- **Response Headers:** HTTP/1.1 200 OK
Date: Wed, 23 Sep 2026 08:20:20 GMT
Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips mod_fcgid/2.3.9 PHP/7.4.23
Last-Modified: Mon, 26 Jan 2026 02:57:57 GMT
ETag: "b83-64941ac9215e3"
Accept-Ranges: bytes
Content-Length: 2947
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/css
- **Response Body:** .wpcf7 .screen-reader-response {
position: absolute;
overflow: hidden;
clip: rect(1px, 1px, 1px, 1px);
clip-path: inset(50%);
height: 1px;
width: 1px;
margin: -1px;
padding: 0;
border: 0;
word-wrap: normal !important; } And rest of the css file

3. Initiator and Purpose

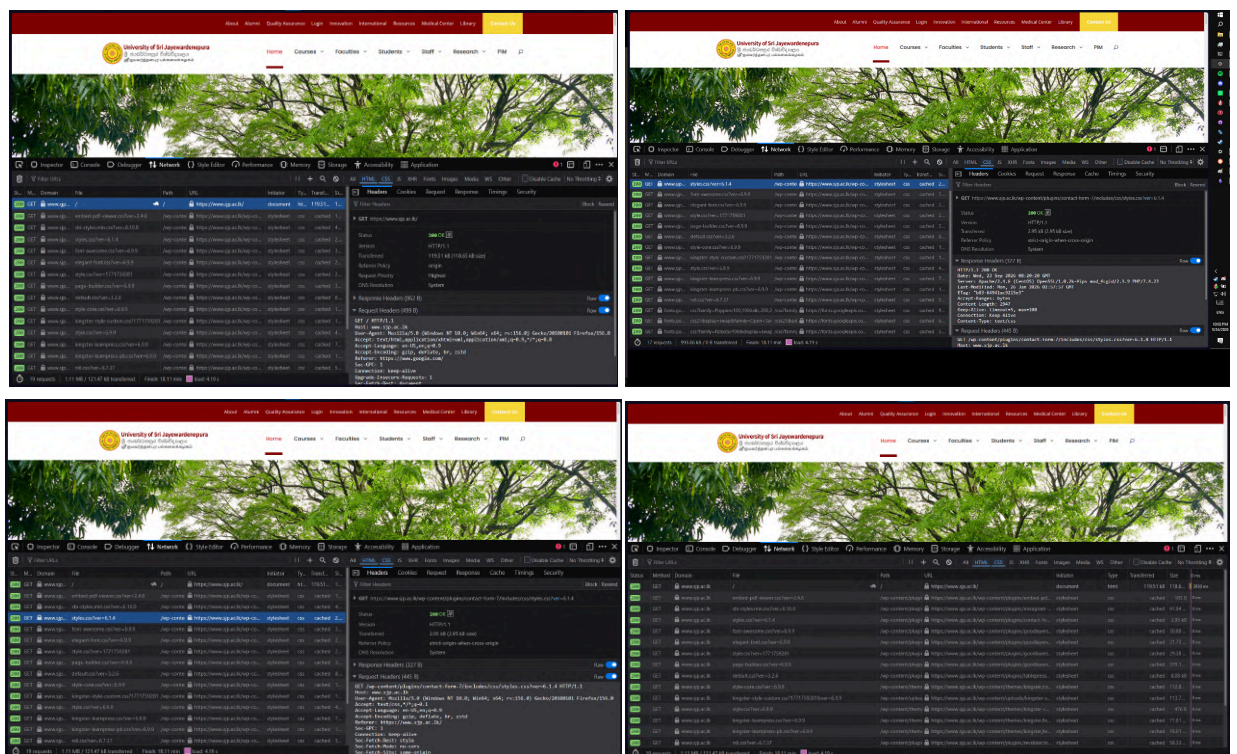
- **Initiator:** stylesheet
- **Purpose:** CSS file to style the page.

4. Security & HTTPS Protection

- **Protocol:** HTTPS
- **Security state:** Secure
- **HTTP version:** HTTP/1.1 for the selected University website request. The HAR data records the security state as **secure**.
- **Server:** Apache with OpenSSL, as indicated by the **Server** response header.

The **https://** scheme indicates that the communication is protected using **TLS (Transport Layer Security)**. TLS encrypts the data exchanged between the browser and the server, helping prevent third parties from reading or modifying the communication while it is being transmitted. It also uses a digital certificate to authenticate the server, helping the browser verify that it is communicating with the website.

1.3 Evidence (Screenshots)



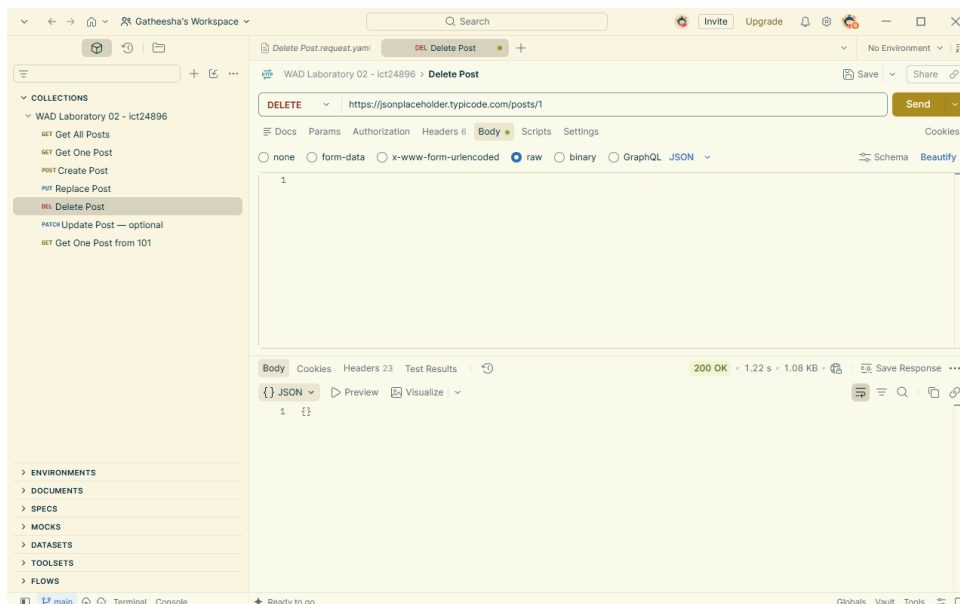
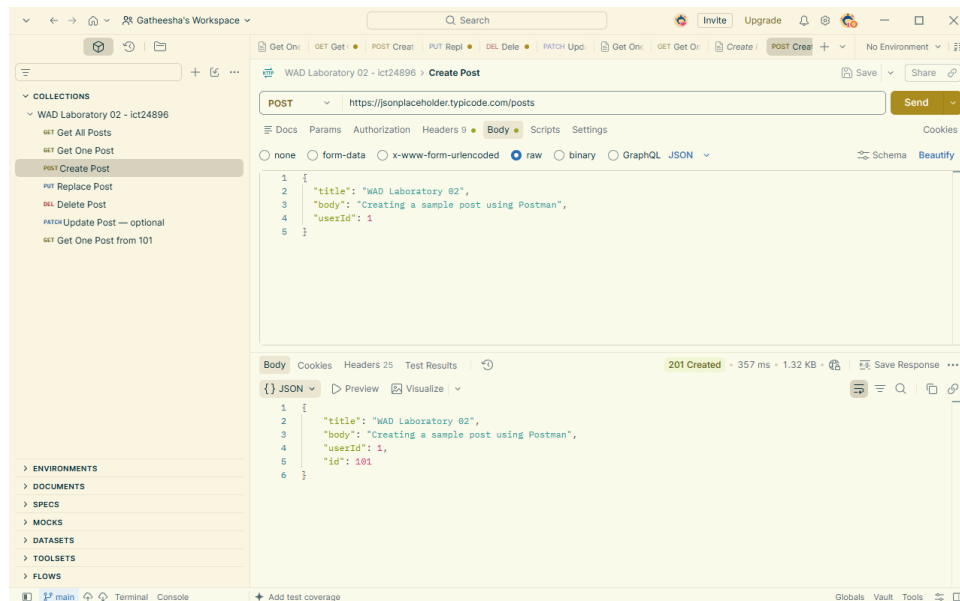
Task 2: Create and Execute an API Test Collection

Postman Collection Name: WAD Laboratory 02 - ict24896

2.1 API Test Results Table

 **WAP-Lab2-Tables** (Postman Observation Table)

2.2 Evidence (Screenshots)



WAD Laboratory 02 - lct24896 > **Get All Posts**

GET <https://jsonplaceholder.typicode.com/posts> Send

Docs Params Authorization Headers 6 **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1
```

Body Cookies Headers 25 Test Results 200 OK · 133 ms · 8.02 KB Save Response

```
1 {
2   {
3     "userId": 1,
4     "id": 1,
5     "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6     "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas tota\nnostrum rerum est autem sunt rem eveniet architecto"
7   },
8 }
9 {
10  "userId": 1,
11  "id": 2,
12  "title": "qui est esse",
13  "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nnulla aperiam non debitis possimus qui neque
```

WAD Laboratory 02 - lct24896 > **Get One Post**

GET <https://jsonplaceholder.typicode.com/posts/1> Send

Docs Params Authorization Headers 6 **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1
```

Body Cookies Headers 25 Test Results 200 OK · 105 ms · 1.31 KB Save Response

```
1 {
2   {
3     "userId": 1,
4     "id": 1,
5     "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6     "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas tota\nnostrum rerum est autem sunt rem eveniet architecto"
7   },
8 }
```

WAD Laboratory 02 - lct24896 > **Replace Post**

PUT <https://jsonplaceholder.typicode.com/posts/1> Send

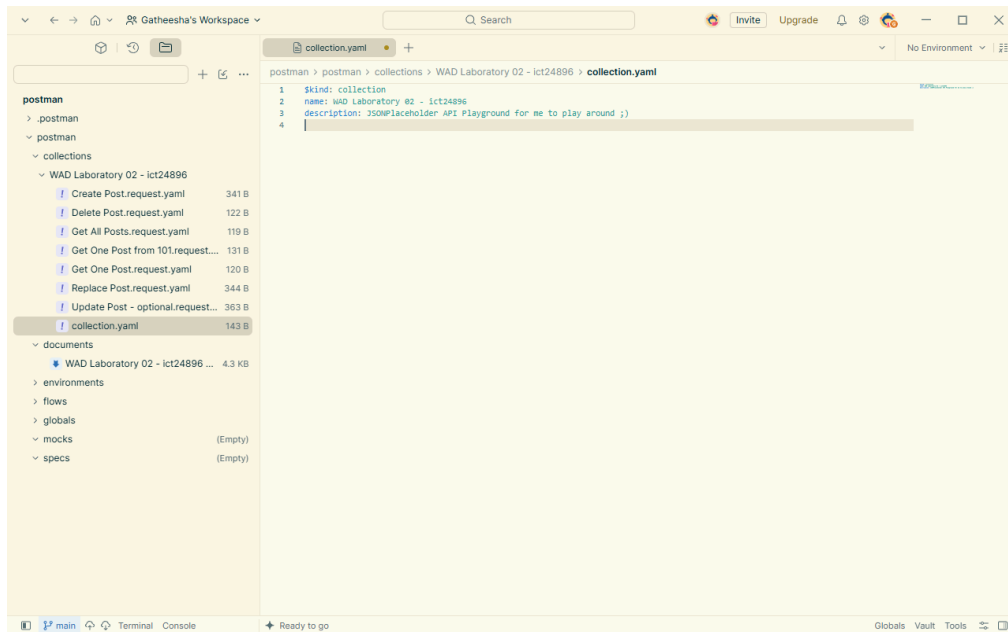
Docs Params Authorization Headers 9 **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```
1 {
2   {
3     "title": "Replaced WAD post",
4     "body": "This resource has been fully replaced",
5     "userId": 1
6   },
7 }
```

Body Cookies Headers 24 Test Results 200 OK · 346 ms · 1.2 KB Save Response

```
1 {
2   {
3     "title": "Replaced WAD post",
4     "body": "This resource has been fully replaced",
5     "userId": 1,
6     "id": 1
7   },
8 }
```



Task 3 - Analyse and Verify

3.1

A browser automatically adds headers and behaviour such as cookies, User-Agent, Accept, Origin/Referer and CORS requests. A request built manually in an API client such as Postman contains only what I set myself. This makes it cleaner and easier to control, but it fails if I forget a required header or authentication token. Browsers also has CORS restrictions, whereas API clients does not.

3.2

- **GET** reads a resource. It is safe and idempotent and has no request body.
- **POST** creates a resource or triggers an action. It is not idempotent, so repeating it can create duplicates.
- **PUT** replaces a whole resource. It is idempotent, and fields left out may be reset or removed.
- **PATCH** partially updates a resource, sending only the fields that change.
- **DELETE** removes a resource. It is idempotent because deleting again leaves the same end state.

3.3

The status code gives the outcome like 2xx success, 4xx client error, 5xx server error etc. The body confirms what actually happened, the returned data, or error messages. Some APIs return 200 OK with an error message in the body, and 201 or 204 responses may have different or empty bodies. A status code alone can be misleading, and a body alone does not show if the request was successful.

3.4

- **Content-Type** describes the format of the body being sent (ex: application/json).
- **Accept** tells the server which response formats the client can handle.
- **Authorization** carries credentials (for example a Bearer token) that identify the client.

3.5

- **GET /posts** returned a JSON array of 100 post objects.
- **GET /posts/1** returned a single post object.
- **POST /posts** returned the submitted fields plus a generated id of 101, with 201 Created and a Location header.
- **PUT /posts/1** returned the full replacement object with id 1.
- **PATCH /posts/1** returned the full post with the updated fields.
- **DELETE /posts/1** returned an empty object {}.

3.6 AI explanation

AI used: Claude

Prompt: *Below is an HTTP request and response captured in Postman. Please explain what happened, including what each header, the status code, and the response body mean.*

Request:

POST https://jsonplaceholder.typicode.com/posts

Content-Type: application/json

{"title":"WAD Laboratory 02","body":"Creating a sample post using Postman","userId":1}

Response:

201 Created

Content-Type: application/json; charset=utf-8

Location: https://jsonplaceholder.typicode.com/posts/101

Content-Length: 112

{"title":"WAD Laboratory 02","body":"Creating a sample post using Postman","userId":1,"id":101}

Response Summary (AI)

- POST asks the server to create a new resource. The target is the collection (/posts) because the server, not the client, assigns the new ID.
- Content-Type: application/json tells the server how to parse the request body. Postman also adds hidden headers such as Host, User-Agent and Content-Length.
- 201 Created is more precise than 200 OK and is the standard success code when a POST creates something.
- The Location header gives the URL of the new resource (/posts/101), and the body echoes the submitted data plus the generated id 101 (100 posts were already preloaded).
- Content-Length: 112 is larger than the roughly 95 visible characters

because the real body is pretty-printed with line breaks and indentation.

- JSONPlaceholder is a fake API, so a later GET /posts/101 would return 404.

Observation

The AI explained why a create request is sent to the collection endpoint (/posts) rather than /posts/101: the server assigns the ID and reports it through the 201 status, the Location header and the id in the body. This connects directly to my results for questions 2 and 5, where POST returned id 101 and a Location header, unlike PUT and PATCH, which addressed the existing post 1.

AI Statement Verification:

Statement: "If you now send GET /posts/101, you'll get a 404."

I sent GET <https://jsonplaceholder.typicode.com/posts/101> in Postman and it returned **404 Not Found**. This confirms the AI's claim and shows that a 201 Created response does not prove the data was stored; the follow-up GET was needed to check.